ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

# Ingeniería Informática + ADE

Universidad de Granada (UGR)

**Autor:** Ismael Sallami Moreno

**Asignatura:** Examenes Anteriores del 2º Parcial SCD

# Índice

**1. Enunciado Carlos Ureña 2014-2015** **3**
1.1. Ejercicio 1. Corrección de errores (6 puntos) . . . . . 3
1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos) . . . . . 3
1.3. Plantilla . . . . . 4

**2. Solución Carlos Ureña 2014-2015** **7**

# 1 Enunciado Carlos Ureña 2014-2015

**Asignatura:** Sistemas Concurrentes y Distribuidos.
**Año Académico:** 2014/2015.
**Grado:** Doble Grado en Ingeniería Informática y ADE + Matemáticas.
**Descripción:** Examen correspondiente a la práctica 3 de SCD.
**Profesor:** Carlos Ureña.
**Fecha:** 12 de enero de 2015.

## 1.1. Ejercicio 1. Corrección de errores (6 puntos)

Considerar el código del archivo disponible en la subsection de plantilla de este documento o aquí, que intenta plantear una solución al problema de la cena de los filósofos con camarero. Hay diversos errores en el código proporcionado que deberás solucionar:

1. Intenta compilar el código proporcionado. Comprueba que hay diversos errores básicos de sintaxis que impiden que el código genere un ejecutable. Encuentra dichos errores y arréglalos hasta que el código compile correctamente. Indica en el folio del examen el número de línea y cómo debería quedar ésta una vez solventado el problema.

2. Ejecuta el programa y observa la salida que genera. El algoritmo presenta un error en su diseño. Explica brevemente en el folio del examen por qué la salida proporcionada no es correcta.

3. Observa el código del programa ejecutado. Encuentra y corrige el error en el diseño del algoritmo que impide que el programa muestre una salida coherente. Señala en el folio del examen en qué línea está el problema y cómo debería quedar ésta para solucionar el error.

4. Se desea modificar el algoritmo dado del problema de los filósofos con camarero para dar servicio a 7 filósofos y 7 tenedores en lugar de a 5. También se desea cambiar el identificador de los procesos para que los filósofos sean los procesos con rank 0, 1, 2, ..., 6 y los tenedores sean los procesos 7, 8, ..., 13. Describe brevemente qué cambios debes realizar en el código proporcionado y en qué líneas.

## 1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos)

Partiendo del algoritmo desarrollado durante la Práctica 3 para resolver el problema de múltiples Productores-Consumidores usando paso de mensajes MPI, se pide realizar los siguientes cambios:

- El valor que los procesos productores que envían al buffer debe ser un valor aleatorio entre 0 y 9 en lugar de uno secuencial como hasta ahora.

- Así mismo, el productor deberá enviar un mensaje al buffer pidiendo permiso para enviar el valor, y el buffer deberá responder al productor autorizando a enviar el mensaje antes de hacerlo (al igual que ya hace el consumidor).

Realiza los cambios necesarios para que, en el caso de que en el buffer se almacenen dos valores iguales consecutivos (independientemente del productor que los envíe), el buffer finalice.

- Antes de finalizar deberá avisar a todos los productores y consumidores para que estos finalicen enviando un valor especial en los mensajes de autorización (por ejemplo, un valor negativo). Sólo cuando todos los procesos hayan finalizado el buffer terminará.

- Implementa esta condición de forma que cuando cualquier proceso vaya a finalizar, imprima un mensaje en pantalla informando de ello.

## 1.3. Plantilla

```
#include <iostream>
#include <time.h>
#include <unistd.h>
#include <stdlib.h>
#include <mpi.h>

#define NUM_FILOSOFOS 5
#define NUM_PROCESOS (2*(NUM_FILOSOFOS)+1)
#define RANK_CAMARERO ((NUM_PROCESOS)-1)

#define TAG_SENTARSE 0
#define TAG_LEVANTARSE 1

using namespace std;
void Filosofo(int id);
void Tenedor(int id);
void Camarero();

// ----------------------------------------------------------------------

void retraso_aleatorio()
{
    static bool primera = true;

    // inicializar generador de números aleatorios (la primera vez)
    if (primera)
    {
        srand(time(NULL));
        primera = false;
    }
    // retraso aleatorio, de entre una y dos décimas de segundo
    usleep(1000L * (100L + rand() % 100L));
}

// ----------------------------------------------------------------------
```

```

int main(int argc, char **argv)
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != NUM_PROCESOS)
    {
        if (rank == 0)
            cout << "El numero de procesos debe ser " << NUM_PROCESOS << endl << flush;
        MPI_Finalize();
        return 0;
    }

    if (rank == RANK_CAMARERO)
        Camarero(); // el 10 es el camarero
    else if ((rank % 2) == 0)
        Filosofo(rank); // los pares (0,2,4,6,8) son Filosofos
    else
        Tenedor(rank); // los impares (1,3,5,7,9) son Tenedores

    MPI_Finalize();
    return 0;
}

// ---------------------------------------------------------------------

void Filosofo(int id)
{
    const int
        primero = (id + 1) % (NUM_PROCESOS - 1),
        segundo = (id + (NUM_PROCESOS - 2)) % (NUM_PROCESOS - 1);

    while (true)
    {
        // solicita sentarse
        cout << "Filosofo " << id << " solicita sentarse." << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_SENTARSE, MPI_COMM_WORLD);

        // solicita y coge primer tenedor
        cout << "Filosofo " << id << " solicita primer tenedor: " << primero << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);
        cout << "Filosofo " << id << " coge primer tenedor: " << primero << endl << flush;

        // solicita coge segundo tenedor
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, segundo, MPI_COMM_WORLD);
```

```
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo << endl << flush;

        // come
        cout << "Filosofo " << id << " COMIENDO" << endl << flush;
        retraso_aleatorio();

        // suelta el segundo tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << segundo << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);

        // suelta el primer tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << primero << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD);

        // solicita levantarse
        cout << "Filosofo " << id << " solicita levantarse." << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_LEVANTARSE, MPI_COMM_WORLD);

        // piensa
        cout << "Filosofo " << id << " PENSANDO" << endl << flush;
        retraso_aleatorio();
    }
}

// ----------------------------------------------------------------------

void Tenedor(int id)
{
    int valor;
    MPI_Status status;

    while (true)
    {
        // espera un mensaje desde cualquier filosofo vecino
        cout << "Tenedor " << id << " espera petición." << endl << flush;
        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, id, MPI_COMM_WORLD, &status);
        cout << "Tenedor " << id << " recibe petición de " << status.MPI_SOURCE << endl << flush;

        // espera a que el filosofo suelte el tenedor
        MPI_Recv(&valor, 1, MPI_INT, status.MPI_SOURCE, id, MPI_COMM_WORLD, &status);
        cout << "Tenedor " << id << " recibe liberación de " << status.MPI_SOURCE << endl << flush;
    }
}

// ----------------------------------------------------------------------

void Camarero()
```

```
133 {
134     int valor, num_sentados = 0, tag_aceptada;
135     MPI_Status status;
136 
137     while (true)
138     {
139         if (num_sentados < NUM_FILOSOFOS - 1)
140             tag_aceptada = MPI_ANY_TAG;
141         else
142             tag_aceptada = TAG_LEVANTARSE;
143 
144         // espera un mensaje desde cualquier filosofo con etiqueta
            aceptable
145         cout << "Camarero espera petición de sentarse/levantarse (sentados
            == " << num_sentados << ")" << endl << flush;
146 
147         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_aceptada,
            MPI_COMM_WORLD, &status);
148         cout << "Camarero recibe petición de filosofo " << status.
            MPI_SOURCE << endl << flush;
149 
150         if (status.MPI_TAG == TAG_LEVANTARSE)
151             num_sentados--;
152         else
153             num_sentados++;
154     }
155 }
```

# 2 Solución Carlos Ureña 2014-2015

## Ejercicio 1

- Errores:
  - En la línea 84 falta un argumento en la función `MPI_Ssend`. No está puesto el tag del mensaje (debería ser "segundo").
  - En la línea 124 que los dos primeros argumentos están al revés.
- Error en el diseño: podemos ver que tras la ejecución del programa, los filósofos no se levantan de la mesa. El error se encuentra en la línea 101, en el paso del mensaje al camarero para solicitar levantarse, el tag del mensaje es el de sentarse (`TAG_SENTARSE`), en lugar del de levantarse (`TAG_LEVANTARSE`).
- Último apartado:
  - para dar servicio a 7 filósofos y 7 tenedores, se deben cambiar las constantes `NUM_FILOSOFOS` en la línea 7.
  - para ello debemos de cambiar la línea 54 poniendo que el rank debe de ser menor al número de filósofos.

- además, debemos de cambiar la asignación de los tenedores en la línea 68 y 69, debido a que la presente no es coherente ni consistente desembocando en un error en el que varios filósofos quieren coger el mismo tenedor.

```
1  #include <iostream>
2  #include <time.h>
3  #include <unistd.h>
4  #include <stdlib.h>
5  #include <mpi.h>
6
7  #define NUM_FILOSOFOS 7
8  #define NUM_PROCESOS (2*(NUM_FILOSOFOS)+1)
9  #define RANK_CAMARERO ((NUM_PROCESOS)-1)
10
11 #define TAG_SENTARSE 0
12 #define TAG_LEVANTARSE 1
13
14 using namespace std ;
15 void Filosofo( int id );
16 void Tenedor ( int id );
17 void Camarero() ;
18
19 // ------------------------------------------------------------------
20
21 void retraso_aleatorio()
22 {
23     static bool primera = true ;
24
25     // inicializar generador de numeros aleatorios (la primera vez)
26     if ( primera )
27     {  srand(time(NULL)) ;
28        primera = false ;
29     }
30     // retraso aleatorio, de entre una y dos décimas de segundo
31     usleep( 1000L * (100L + rand() % 100L ) );
32 }
33
34 // ------------------------------------------------------------------
35
36 int main(int argc,char** argv )
37 {
38     int rank,size;
39
40     MPI_Init( &argc, &argv );
41     MPI_Comm_rank( MPI_COMM_WORLD, &rank );
42     MPI_Comm_size( MPI_COMM_WORLD, &size );
43
44     if ( size != NUM_PROCESOS )
45     {
46         if( rank == 0 )
47         cout << "El numero de procesos debe ser " << NUM_PROCESOS << endl
               << flush;
48         MPI_Finalize( );
49         return 0;
50     }
```

```
    if ( rank == RANK_CAMARERO )
        Camarero() ; // el 14 es el camarero
    else if (rank<NUM_FILOSOFOS)
        Filosofo( rank );
    else
        Tenedor( rank );

    MPI_Finalize( );
    return 0;
}

// ----------------------------------------------------------------------

void Filosofo( int id )
{
    const int
        primero = NUM_FILOSOFOS + id ,
        segundo = NUM_FILOSOFOS + ((id+1)%NUM_FILOSOFOS) ;

    while ( true )
    {
        // solicita sentarse
        cout << "Filosofo " << id << " solicita sentarse." << endl << flush
            ;
        MPI_Ssend( &id, 1, MPI_INT, RANK_CAMARERO, TAG_SENTARSE,
            MPI_COMM_WORLD );

        // solicita y coge primer tenedor
        cout << "Filosofo " << id << " solicita primer tenedor: " <<
            primero << endl << flush;
        MPI_Ssend( &id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD );
        cout << "Filosofo " << id << " coge primer tenedor: " << primero <<
             endl << flush;

        // solicita coge segundo tenedor
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
            << endl << flush;
        MPI_Ssend( &id, 1, MPI_INT,segundo, segundo ,MPI_COMM_WORLD);
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
            << endl << flush;

        // come
        cout << "Filosofo " << id << " COMIENDO" << endl << flush;
        retraso_aleatorio() ;

        // suelta el segundo tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << segundo << endl
             << flush;
        MPI_Ssend( &id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD );

        // suelta el primer tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << primero << endl
             << flush;
        MPI_Ssend( &id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD );

```

```
        // solicita levantarse
        cout << "Filosofo " << id << " solicita sentarse." << endl << flush
            ;
        MPI_Ssend ( &id, 1, MPI_INT, RANK_CAMARERO, TAG_LEVANTARSE,
            MPI_COMM_WORLD );

        // piensa
        cout << "Filosofo " << id << " PENSANDO" << endl << flush ;
        retraso_aleatorio() ;
    }
}

// ----------------------------------------------------------------------

void Tenedor( int id )
{
    int valor ;
    MPI_Status status;

    while ( true )
    {
        // espera un mensaje desde cualquier filosofo vecino
        cout <<"Tenedor " << id << " espera petición." << endl << flush;
        MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, id, MPI_COMM_WORLD, &
            status );
        cout << "Tenedor " << id << " recibe petición de " << status.
            MPI_SOURCE << endl << flush ;

        // espera a que el filosofo suelte el tenedor
        MPI_Recv( &valor, 1, MPI_INT, status.MPI_SOURCE, id, MPI_COMM_WORLD
            , &status );
        cout<<"Tenedor " << id << " recibe liberación de " << status.
            MPI_SOURCE << endl << flush;
    }
}

// ----------------------------------------------------------------------

void Camarero()
{
    int valor, num_sentados = 0, tag_aceptada ;
    MPI_Status status;

    while ( true )
    {
        if ( num_sentados < NUM_FILOSOFOS-1 )
            tag_aceptada = MPI_ANY_TAG ;
        else
            tag_aceptada = TAG_LEVANTARSE ;

        // espera un mensaje desde cualquier filosofo con etiqueta
            aceptable
        cout << "Camarero espera petición de sentarse/levantarse (sentados
            == " << num_sentados << ")" << endl << flush;

        MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_aceptada,
```

```
            MPI_COMM_WORLD, &status);
        cout << "Camarero recibe petición de filosofo " << status.
            MPI_SOURCE << endl << flush ;

        if ( status.MPI_TAG == TAG_LEVANTARSE )
            num_sentados -- ;
        else
             num_sentados ++ ;
    }
}
```

La solución en código se encuentra aqui.

## Ejercicio 2

En la resolución de este ejercicio usamos esta (plantilla)(pincha en plantilla para acceder a ella).

- Hemos añadido en la línea 58 en la función de producir valor esta línea **int valor = aleatorio<0, 9>();** para que produzca el valor aleatorio.
- En la función del productor debemos añadir **MPI_Ssend(&peticion, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);** para que el productor pida permiso para enviar el valor y añadimos en el **switch** que cuando sea el caso del productor reciba la petición de aviso: **MPI_Recv(&peticion_productor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_productor, MPI_COMM_WORLD, &estado);**

Para la solución del apartado 2.2 revisa este código donde los comentarios **apartado2.2** reflejan los cambios que se han realizado.

```
#include <chrono> // duraciones (duration), unidades de tiempo
#include <cstdlib>
#include <iostream>
#include <mpi.h>
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <thread> // this_thread::sleep_for

using namespace std;
using namespace std::this_thread;
using namespace std::chrono;

#define TERM -1 // Valor especial para finalizar

const int
    etiq_prod = 1, // Etiqueta para mensajes de productores
    etiq_cons = 2, // Etiqueta para mensajes de consumidores
    num_prod = 4, // Número de productores
    num_cons = 5, // Número de consumidores
    id_ini_productores = 0, // ID inicial de los productores
    id_ini_consumidores = num_prod + 1, // ID inicial de los consumidores
    id_buffer = num_prod, // ID del buffer
    num_procesos_esperado = num_prod + num_cons + 1, // Número total de
        procesos esperados
    num_items = 20, // Número total de ítems a producir/consumir
```

```
    tam_vector = 10; // Tamaño del buffer

/**
 * @brief Función que produce un número aleatorio entre dos valores
 *
 * @tparam min Valor mínimo. Constante conocida en tiempo de compilación
 * @tparam max Valor máximo. Constante conocida en tiempo de compilación
 */
template <int min, int max> int aleatorio() {
    static default_random_engine generador((random_device())());
    static uniform_int_distribution<int> distribucion_uniforme(min, max);
    return distribucion_uniforme(generador);
}
// ---------------------------------------------------------------------
/**
 * @brief Función que simula la producción de un valor entre 0 y 9
 *
 * @note Duerme un tiempo aleatorio entre 10 y 100 milisegundos
 *
 * @return int Valor producido
 */
int producir() {
    sleep_for(milliseconds(aleatorio<10, 100>())); // Simula tiempo de producción
    return aleatorio<0, 9>(); // Devuelve un valor aleatorio entre 0 y 9
}

/**
 * @brief Función que simula la producción de valores por un productor
 *
 * @param orden Orden del productor (global)
 */
void funcion_productor(int orden) {
    for (unsigned int i = 0; i < num_items / num_prod; i++) {
        // producir valor
        int valor_prod = producir();
        int rec;
        MPI_Status estado;
        // enviar valor
        cout << "Productor va a enviar valor " << valor_prod << endl << flush;
        MPI_Ssend(&valor_prod, 1, MPI_INT, id_buffer, etiq_prod, MPI_COMM_WORLD); // Enviar valor al buffer
        MPI_Recv(&rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD, &estado); // Recibir confirmación del buffer
        if (rec == TERM) { // Si recibe el valor especial, termina
            break;
        }
        MPI_Ssend(&valor_prod, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD); // Enviar valor al buffer
    }
    cout << "Termina productor " << orden << endl; // Mensaje de finalización del productor
}
// ---------------------------------------------------------------------

```

```
/**
 * @brief Función que simula la consumición de un valor
 *
 * @param valor_cons Valor a consumir
 */
void consumir(int valor_cons) {
    // espera bloqueada
    sleep_for(milliseconds(aleatorio<110, 200>())); // Simula tiempo de consumo
    cout << "Consumidor ha consumido valor " << valor_cons << endl << flush;
}
// ----------------------------------------------------------------------

/**
 * @brief Función que simula la consumición de valores por un consumidor
 *
 * @param orden Orden del consumidor (global)
 */
void funcion_consumidor(int orden) {
    int peticion, valor_rec = 1;
    MPI_Status estado;

    for (unsigned int i = 0; i < num_items / num_cons; i++) {
        MPI_Ssend(&peticion, 1, MPI_INT, id_buffer, etiq_cons, MPI_COMM_WORLD); // Enviar petición al buffer
        MPI_Recv(&valor_rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD, &estado); // Recibir valor del buffer
        if (valor_rec == TERM) { // Si recibe el valor especial, termina
            break;
        }
        cout << "Consumidor ha recibido valor " << valor_rec << endl << flush;
        consumir(valor_rec); // Consumir el valor recibido
    }
    cout << "Termina consumidor " << orden << endl; // Mensaje de finalización del consumidor
}

/**
 * @brief Función que simula un buffer de tamaño tam_vector
 */
void funcion_buffer() {
    int buffer[tam_vector],      // buffer con celdas ocupadas y vacías
        valor,                   // valor recibido o enviado
        primera_libre = 0,       // índice de primera celda libre
        primera_ocupada = 0,     // índice de primera celda ocupada
        num_celdas_ocupadas = 0; // número de celdas ocupadas
    MPI_Status estado;           // metadatos del mensaje recibido
    bool terminar = false;       // bandera para finalizar

    for (unsigned int i = 0; i < num_items * 2 && !terminar; i++) {
        // 1. determinar si puede enviar solo prod., solo cons, o todos
        int etiq;

        if (num_celdas_ocupadas == 0)               // si buffer vacío
```

```
            // solo prods
            etiq = etiq_prod;
        else if (num_celdas_ocupadas == tam_vector) // si buffer lleno
            // solo cons
            etiq = etiq_cons;
        else                                        // si no vacío ni lleno
            // cualquiera
            etiq = MPI_ANY_TAG;

        // 2. recibir un mensaje del emisor o emisores aceptables

        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq, MPI_COMM_WORLD,
                 &estado); // Recibir mensaje del productor o consumidor

        // 3. procesar el mensaje recibido

        switch (estado.MPI_TAG) { // leer emisor del mensaje en metadatos

            case etiq_prod: // si ha sido el productor: insertar en buffer
                // Da permiso
                MPI_Ssend(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
                    MPI_COMM_WORLD); // Enviar confirmación al productor
                MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
                    MPI_COMM_WORLD, &estado); // Recibir valor del productor

                // Dos valores iguales consecutivos recibidos
                if (valor == buffer[(primera_libre - 1 + tam_vector) %
                    tam_vector]) {
                    cout << "Empezamos a terminar el programa" << endl;
                    terminar = true;
                    int aux;
                    MPI_Request request;
                    // terminar procesos productores
                    for (int i = 0; i < id_buffer; i++) {
                        MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
                            MPI_COMM_WORLD, &request); // Recibir cualquier
                            mensaje pendiente
                        MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
                            MPI_COMM_WORLD, &request); // Recibir cualquier
                            mensaje pendiente
                        aux = TERM;
                        MPI_Ssend(&aux, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
                            // Enviar valor especial para finalizar
                    }

                    // terminar procesos consumidores
                    for (int i = id_buffer + 1; i < num_procesos_esperado;
                        i++) {
                        MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
                            MPI_COMM_WORLD, &request); // Recibir cualquier
                            mensaje pendiente
                        aux = TERM;
                        MPI_Ssend(&aux, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
                            // Enviar valor especial para finalizar
                    }
                }
```

```
                if (!terminar) {
                    buffer[primera_libre] = valor; // Insertar valor en el buffer
                    primera_libre = (primera_libre + 1) % tam_vector; // Actualizar índice de primera celda libre
                    num_celdas_ocupadas++; // Incrementar número de celdas ocupadas
                    cout << "Buffer ha recibido valor " << valor << endl;
                }
                break;

            case etiq_cons: // si ha sido el consumidor: extraer y enviarle
                if (!terminar) {
                    valor = buffer[primera_ocupada]; // Extraer valor del buffer
                    primera_ocupada = (primera_ocupada + 1) % tam_vector; // Actualizar índice de primera celda ocupada
                    num_celdas_ocupadas--; // Decrementar número de celdas ocupadas
                    cout << "Buffer va a enviar valor " << valor << endl;
                    MPI_Ssend(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0, MPI_COMM_WORLD); // Enviar valor al consumidor
                }
                break;
        }
    }
    cout << "Termina buffer" << endl; // Mensaje de finalización del buffer
}

// ----------------------------------------------------------------------

int main(int argc, char *argv[]) {
    int id_propio, num_procesos_actual;

    if (num_items % num_prod != 0 || num_items % num_cons != 0) {
        cout << "Numero de productores y consumidores no valido" << endl;
        exit(1);
    }

    // inicializar MPI, leer identif. de proceso y número de procesos
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);

    if (num_procesos_esperado == num_procesos_actual) {
        // ejecutar la operación apropiada a 'id_propio'
        if (id_ini_productores <= id_propio && id_propio < id_ini_productores + num_prod)
            funcion_productor(id_propio - id_ini_productores); // Ejecutar función productor
        else if (id_propio == id_buffer)
            funcion_buffer(); // Ejecutar función buffer
        else
            funcion_consumidor(id_propio - id_ini_consumidores); // Ejecutar función consumidor
    } else {
```

```
        if (id_propio == 0) // solo el primero escribe error, indep. del
            rol
        {
            cout << "el número de procesos esperados es:    "
                 << num_procesos_esperado << endl
                 << "el número de procesos en ejecución es: "
                 << num_procesos_actual << endl
                 << "(programa abortado)" << endl;
        }
    }

    // al terminar el proceso, finalizar MPI
    MPI_Finalize();
    return 0;
}
```

Pincha aqui para acceder al fichero.